

Fundamentos de Sistemas Robóticos com ROS 2

Etapa 1

Professor Mario Pego

mario.peggo@prof.infnet.edu.br



Cronograma



Fundamentos de Sistemas Robóticos com ROS 2

<i>Etapas</i>	<i>Segunda-feira/Quarta-feira</i>
Etapa 1	20/07/26 e 22/07/26
Etapa 2/Leitura TP1	27/07/26 e 29/07/26
Etapa 3	03/08/26 e 05/08/26
Etapa 4/Entrega do TP1/Leitura TP2	10/08/26 e 12/08/2026
Etapa 5	17/08/26 e 19/08/26
Etapa 6/Entrega TP2/Leitura do TP3	24/08/26 e 26/08/26
Etapa 7	31/08/26 e 02/09/26
Etapa 8/Entrega TP3	07/09/26 (FERIADO) e 09/09/26
Etapa 9/Leitura do AT	14/09/26 e 16/09/26
Entrega Assessment	21/09/26 e 23/09/26
Reentrega	28/09/26 e 30/09/26

Histórico

As Origens dos Robôs

-1250

Bispo Albertus Magnus ofereceu um banquete no qual os convidados foram servidos por serviçais de metal. Vendo isso, São Tomás de Aquino esmagou os atendentes e acusou o bispo de feiticeiro.



1495

Leonardo da Vinci desenhou e provavelmente construiu o primeiro robô humanoide. O robô foi projetado para sentar, mover os braços e mover a cabeça através de um pescoço flexível e, além disso, abrir e fechar a boca.



1640

Descartes construiu um autômato feminino que ele chamou de "Ma fille Francine." Ela acompanhou Descartes em uma viagem mas foi atirada ao mar pelo capitão do navio que pensou que ela fosse um trabalho de Satanás.



1738

Jacques de Vaucanson construiu um pato mecânico feito com mais de 4000 peças. O pato era capaz de emitir som, tomar banho, beber água, comer grãos, digeri-los e evacuar.



1801

1801

Joseph Jacquard inventou uma máquina textil que era controlada por cartões perfurados. (Imagem do Smithsonian NMAH)



1805

Boneca construída por Maillardet capaz de escrever em Inglês e Francês e ainda desenhar paisagens.





1865

John Brainerd criou o Homem a Vapor, aparentemente usado para puxar coisas.



1885

Frank Reade Jr. construiu o Homem Elétrico que é mais ou menos uma versão do Homem a Vapor.



1893

Dr. Achibald Campion constroi o Boilerplate, que é um protótipo de soldado.



1923

Karel Capek inventa o termo Robô (Robot) em sua peça teatral *Rossum's Universal Robots (R.U.R)*. Robô vem da palavra Tcheca *robota*, que significa “servitude, trabalho forçado”.



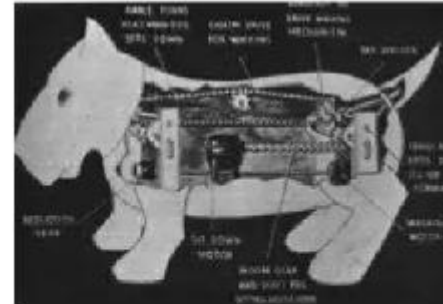
1937/
1938

Westinghouse cria ELEKTRO, um robô com aparência humana que podia andar e falar.



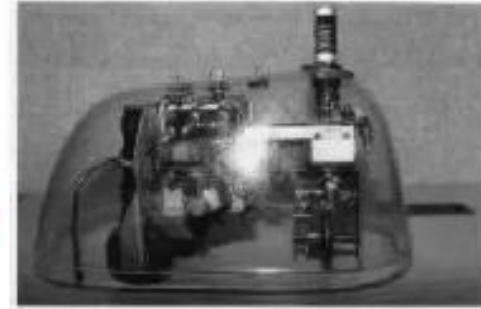
1940

Sparko, um cachorro construído pela Westinghouse, usando componentes mecânicos e elétricos.



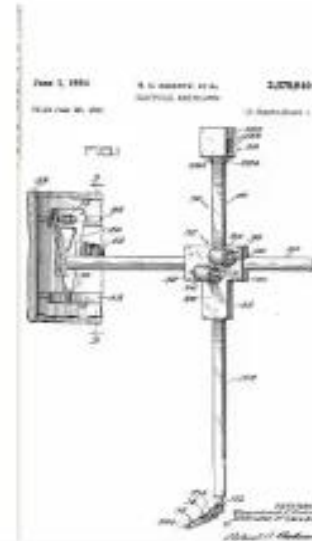
1948/
49

W. Grey Walter cria seus primeiros robôs; Elmer e Elsie conhecidos como as tartarugas robôs. Os robôs eram capazes de encontrar a estação de recarga quando as baterias estavam se tornando fracas.



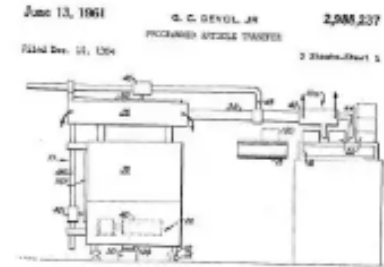
1951

Raymond Goertz projeta o primeiro braço articulado teleoperado para a Comissão de Energia Atômica dos EUA. Este feito é geralmente considerado como o marco principal na tecnologia de controle de força. (US Patent 2679940)



1954

George Devol projeta o primeiro robô programável e o chama de "Universal Automation." (US patent 2 998 237)



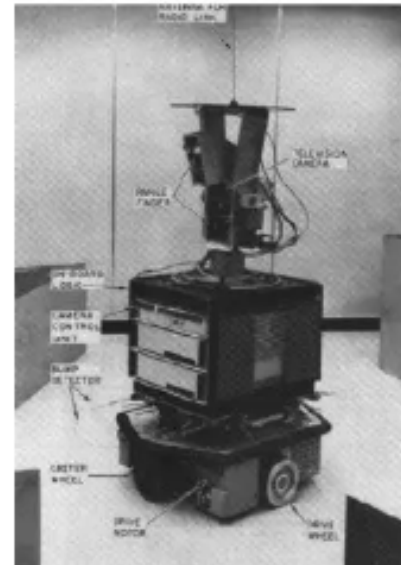
1960's
-1970's

Os robôs industriais são criados. Robotic Industries Association define que **“um robô é um manipulador programável ou reprogramável multifuncional, projetado para mover peças, ferramentas ou dispositivos especiais, através de movimentos programados, possibilitando a realização de uma variedade de tarefas”**.



1956 — Pesquisadores combinam “percepção e capacidade de solução de problemas” usando computadores, câmeras e sensores de tato. A idéia é estudar os tipos de ações inteligentes que esses robôs seriam capazes de realizar. Uma nova disciplina nasceu: IA.

1960 — Shakey é projetado no Stanford Research Institute International. Ele continha uma câmera de televisão, sensores, lógica on-board, unidade de controle da câmera e uma antena de rádio para comunicação. Shakey era controlado por um computador localizado em uma outra sala.





1960

American Machine and Foundry (AMF Corp.) coloca no mercado o primeiro robô cilíndrico chamado Versatran. Ele foi desenhado por Harry Johnson e Veljko Milenkovic.



1961

UNIMATE se torna o primeiro robô industrial em uso. Ele foi usado em uma fábrica da General Motors em New Jersey.





1968 — Marvin Minsky criou o Braço Tentáculo (Tentacle Arm). O braço tentáculo era capaz de levantar uma pessoa.



1969 — Victor Scheinman desenvolve o Stanford Arm, que foi o primeiro robô elétrico de sucesso.





1974 — O Silver Arm foi criado por David Silver. Ele montava pequenas peças usando feedback de sensores de tato e pressão.

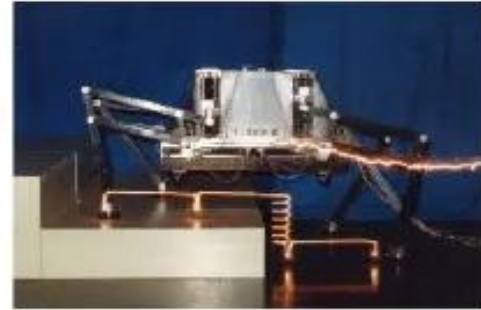


1975 — Victor Schenman desenvolve o Programmable Universal Manipulation Arm (Puma). Ele foi largamente usado em robôs industriais.





1981 — Shigeo Hirose desenvolve Titan II. Ele era um quadrúpede que podia subir escadas. A foto é do Titan III, que foi o sucessor do Titan II.



1982 — A Heathkit Corporation desenvolve Hero Jr. A intenção é que ele fosse um companheiro em casa. Ele tinha um relógio despertador e podia cantar diversas canções. Programas adicionais eram armazenados em 250 cartuchos.





- 1985 — Shigeo Hirose continuou a trabalhar com o TITAN e em 1985 ele completou o TITAN IV que podia andar a uma velocidade de 40 cm/sec.



- 1985 — Criado pela General Robotics Corp. o RB5X era um robô programável. Ele tinha sensores de infravermelho, transmissão remota de áudio e vídeo, sensores de choque e um sintetizador de voz. Ele tinha um software que possibilitava que ele aprendesse o seu meio ambiente.





1988 — O primeiro HelpMate robô começa a trabalhar no Danbury Hospital em Connecticut.



1989 — O Mobile Robots Group do MIT criou Genghis, um robô móvel





1990 — Dr. William Bargar e Howard Pual do Integrated Surgical Systems Inc. e a University of California at Davis desenvolveram o Robodoc. Ele realizava uma hip-replacement operation em um cachorro (1992 em um paciente humano).



1996 — Honda cria P2, o primeiro passo importante na criação do seu ASIMO. P2 é o primeiro robô humanoide bípede auto controlado.



1997 — PathFinder da NASA desce em Marte. Ele é um veículo robótico que captura e envia dados e imagens sobre Marte para a Terra a medida que anda pelo planeta.



1998 — Dr. Cynthia cria Kismet, uma criatura robótica que se interage socialmente com pessoas. Ela usa dicas das pessoas com quem está interagindo como uma base para sua interação.





1998 — O primeiro braço biônico é colocado em Campbell Aird. O braço é chamado Edinburg Modular Arm System (EMAS).

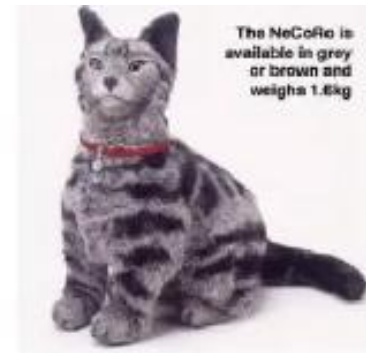


1999 — Sony apresenta o primeiro cachorro eletrônico Aibo.





- 2001 — Omron apresenta seu gato, NeCoRo, como um competidor do Aibo da Sony. Ele vem com a tecnologia Mind and Consciousness (MaC), que possibilita que o gato gere sentimentos.



- 2002 — Honda cria o Advanced Step in Innovative Mobility (ASIMO). Pretende-se que ele seja um assistente pessoal. Ele reconhece a face de seu dono, a voz e o nome. Pode ler email e é capaz de transmitir video de sua câmara para um PC.

<http://asimo.honda.com/>





2005 — O Korean Institute of Science and Technology (KIST), cria HUBO e reivindica que ele é o robô mais inteligente do mundo. Este robô é conectado a um computador através de uma conexão wireless de alta velocidade. O computador realiza todo o raciocínio para o robô.



2002 — Honda cria o Advanced Step in Innovative Mobility (ASIMO). Pretende-se que ele seja um assistente pessoal. Ele reconhece a face de seu dono, a voz e o nome. Pode ler email e é capaz de transmitir vídeo de sua câmara para um PC.

Repetido para comparação com HUBO





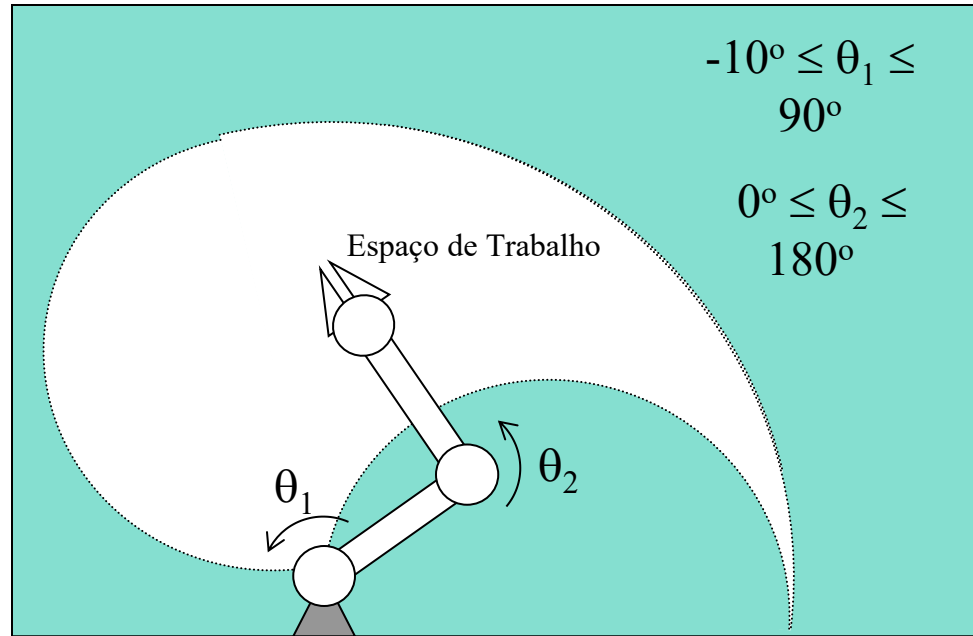
Introdução

Fundamentos da tecnologia de robôs

ROBÓTICA: CONCEITOS BÁSICOS

- **Robô**: Máquina programável de propósito geral que tem existência no mundo físico e atua nele através de movimentos mecânicos.
- **Espaço de Trabalho**: Região do mundo que o robô pode alcançar através dos seus movimentos, onde pode levar a cabo as tarefas programadas.

Exemplo de Robô em seu Espaço de Trabalho



Robô Manipulador Industrial:

- Robotic Industries Association (RIA): “manipulador multifuncional reprogramável, projetado para movimentar materiais, partes, ferramentas ou peças especiais, através de diversos movimentos programados, para o desempenho de uma variedade de tarefas”.
- Norma ISO 10218: “máquina manipuladora, com vários graus de liberdade, controlada automaticamente, reprogramável, multifuncional, que pode ter base fixa ou móvel, para utilização em aplicações de automação industrial”.

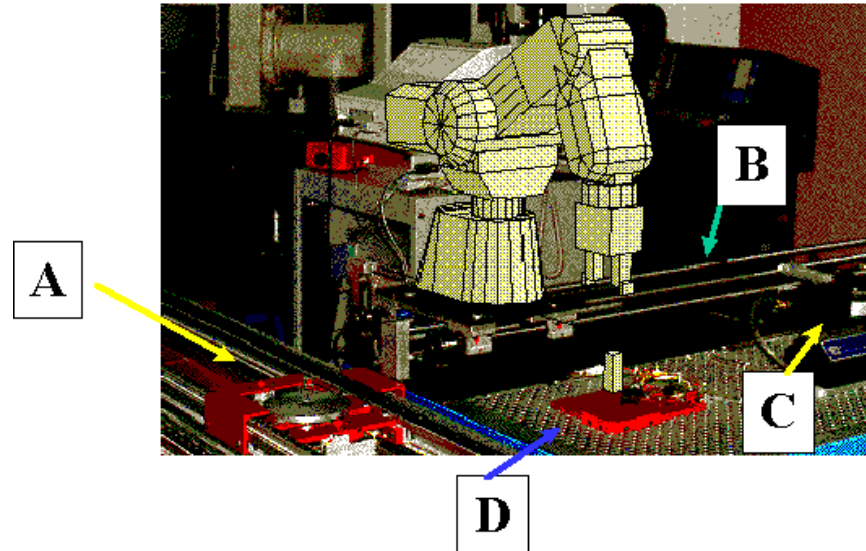
Características Gerais:

- Braço manipulador – emula a função do braço humano: através do seu movimento manipula objetos (ferramentas, peças, etc.) no seu espaço de trabalho.
- Pelo menos um ponto da sua estrutura é fixo na sua base.
- Seu espaço de trabalho é limitado a uma região próxima a esse ponto fixo.



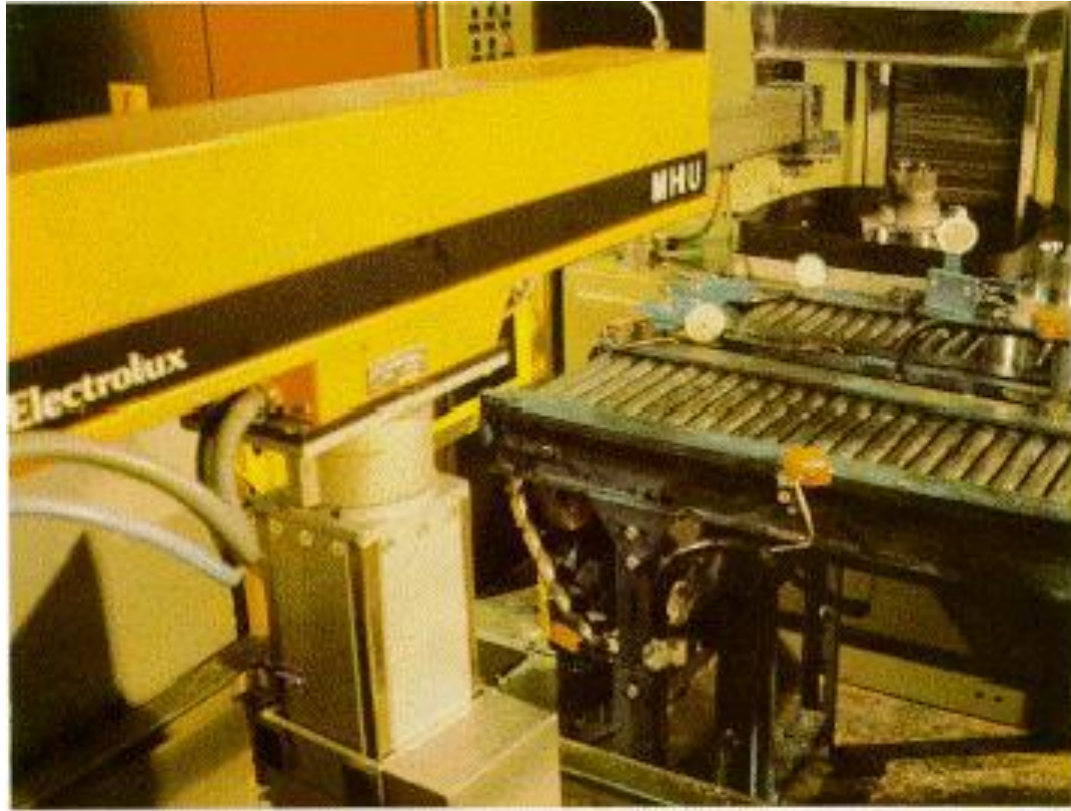
Aplicações de Robôs Manipuladores

- Robôs manipuladores industriais geralmente trabalham integrados em Células de Trabalho, associados a outras máquinas:
 - Máquinas de comando numérico.
 - Veículos de Guiado Automático (AGV's).
 - Sistemas de armazenagem automática (AS/RS - Automatic Storage/Retrieval System)
 - Esteiras.
 - Pontes rolantes.
 - Outros robôs.
 - etc.



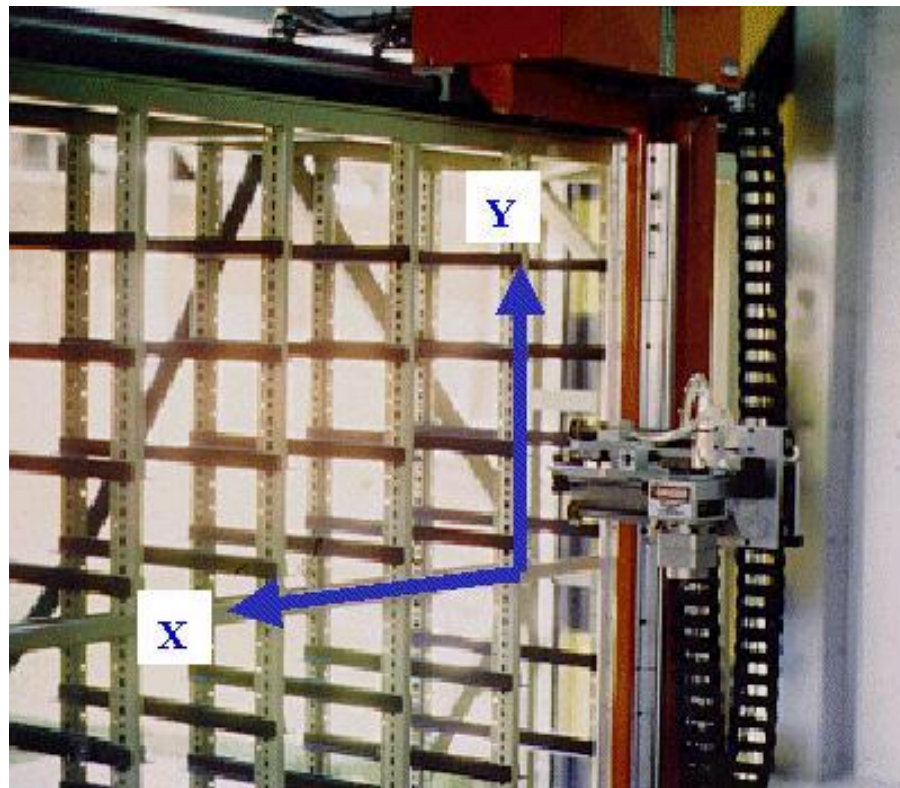
- A) Estação de posicionamento sobre o transportador para carga/descarga.
- B) Eixo transversal para aumentar o espaço de trabalho do robô.
- C) Estação de inspeção por computador integrada ao robô.
- D) Estação de montagem.

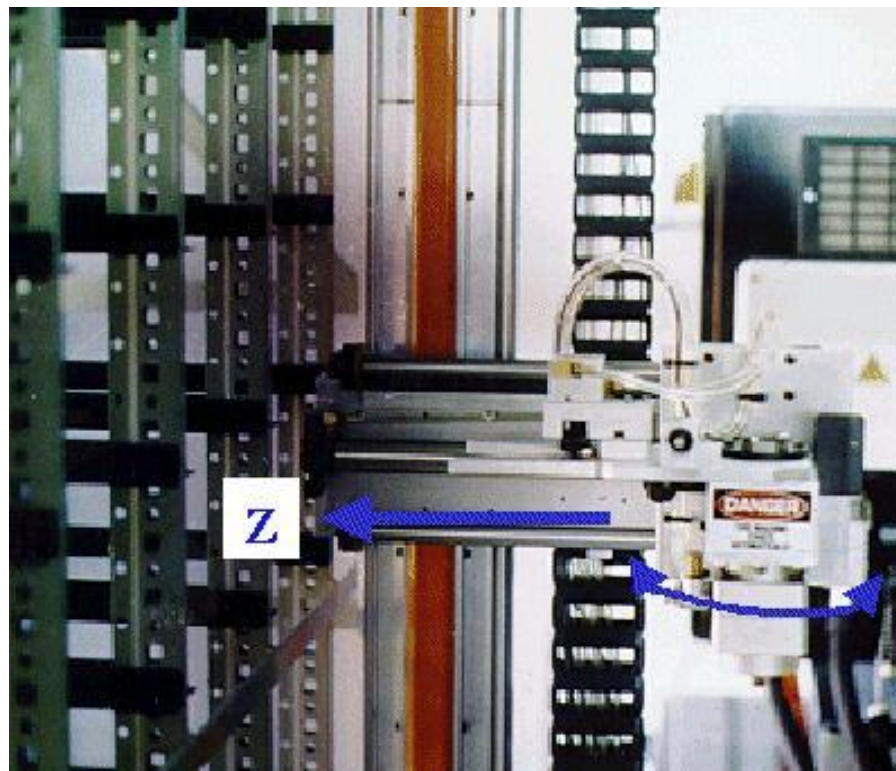
Carga e Descarga (Pick and Place):



Armazenamento Automático (AS/RS):



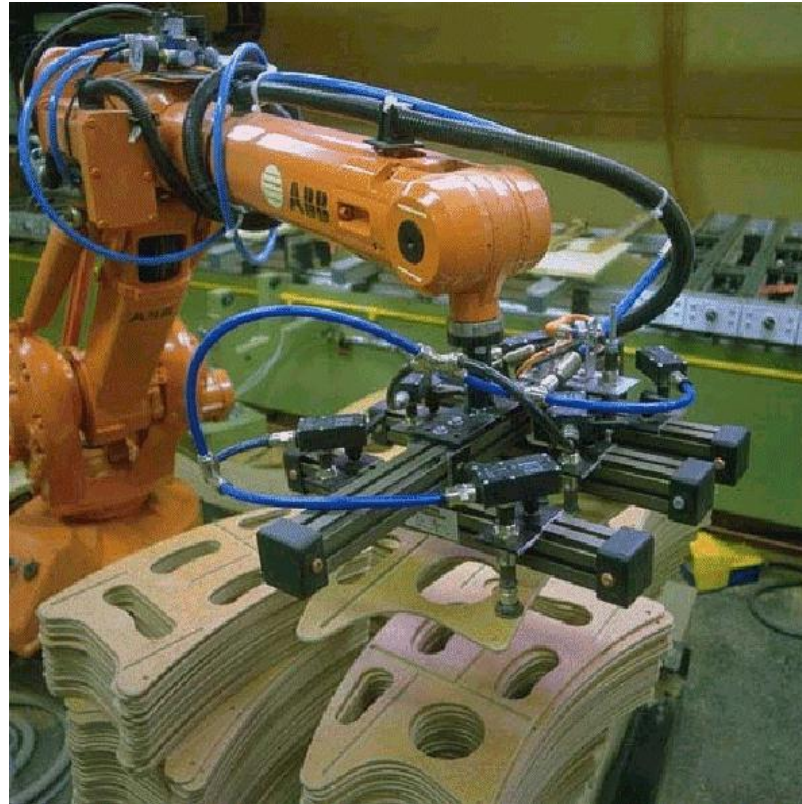




Empacotamento em paletas (pallets):



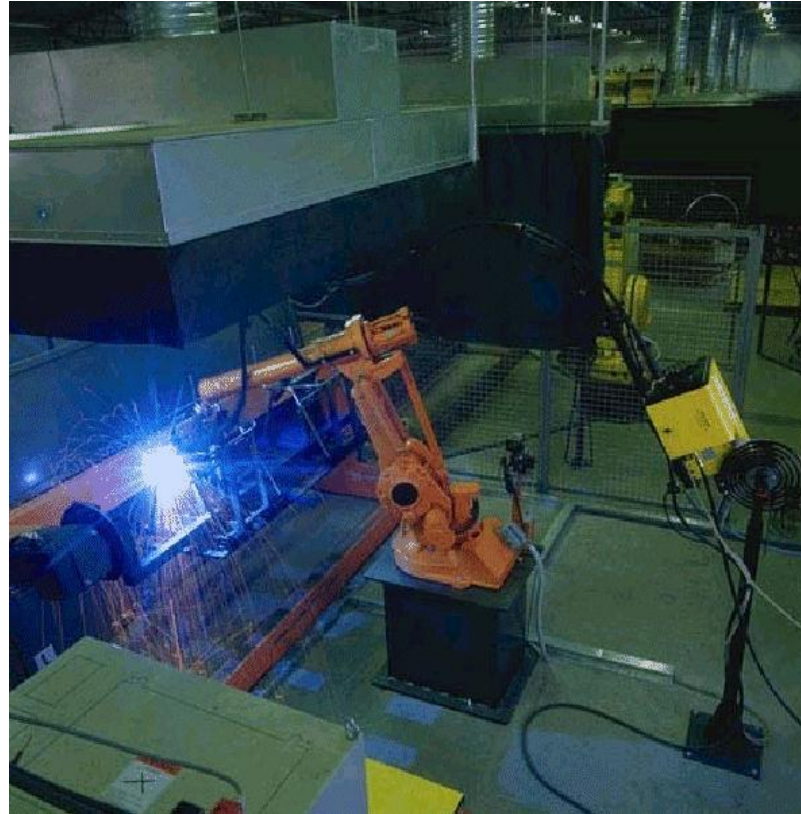
Transporte de materiais:



Montagem automatizada:



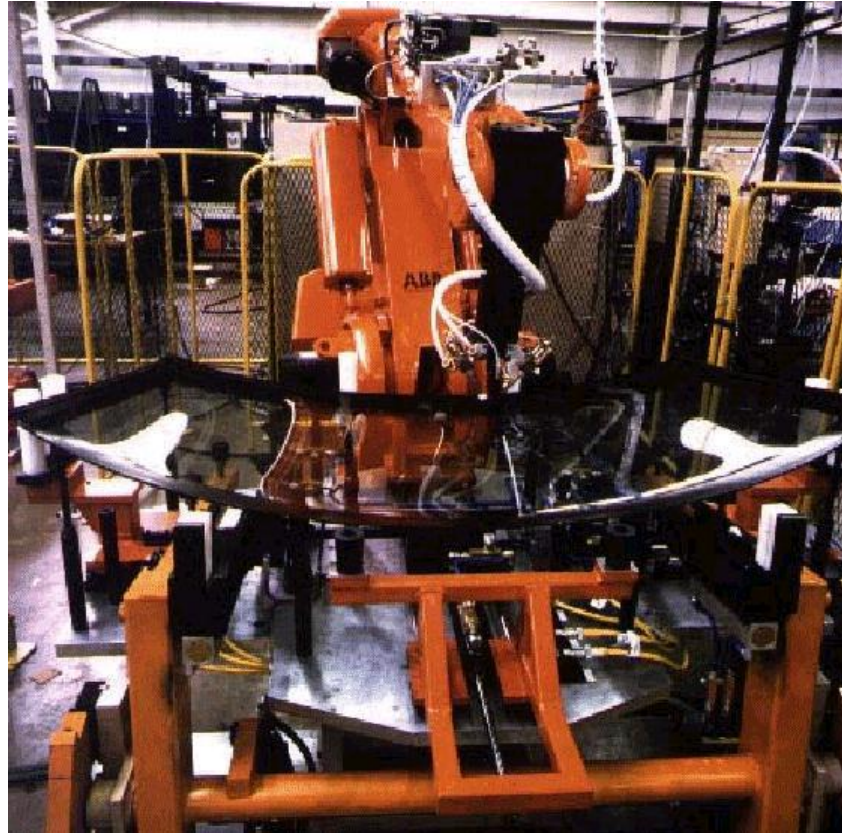
Soldagem:







Aplicação de resina:



Descrever a arquitetura, os
componentes e os casos de
uso do ROS 2

O QUE É ROS ?

Definição

- Robot Operation System
- De fato ele não é um sistema operacional, mas um *framework* para desenvolvimento de software para robótica
- Open source
- Proposta de grande modularidade e compartilhamento

Exemplo de Aplicação



C++

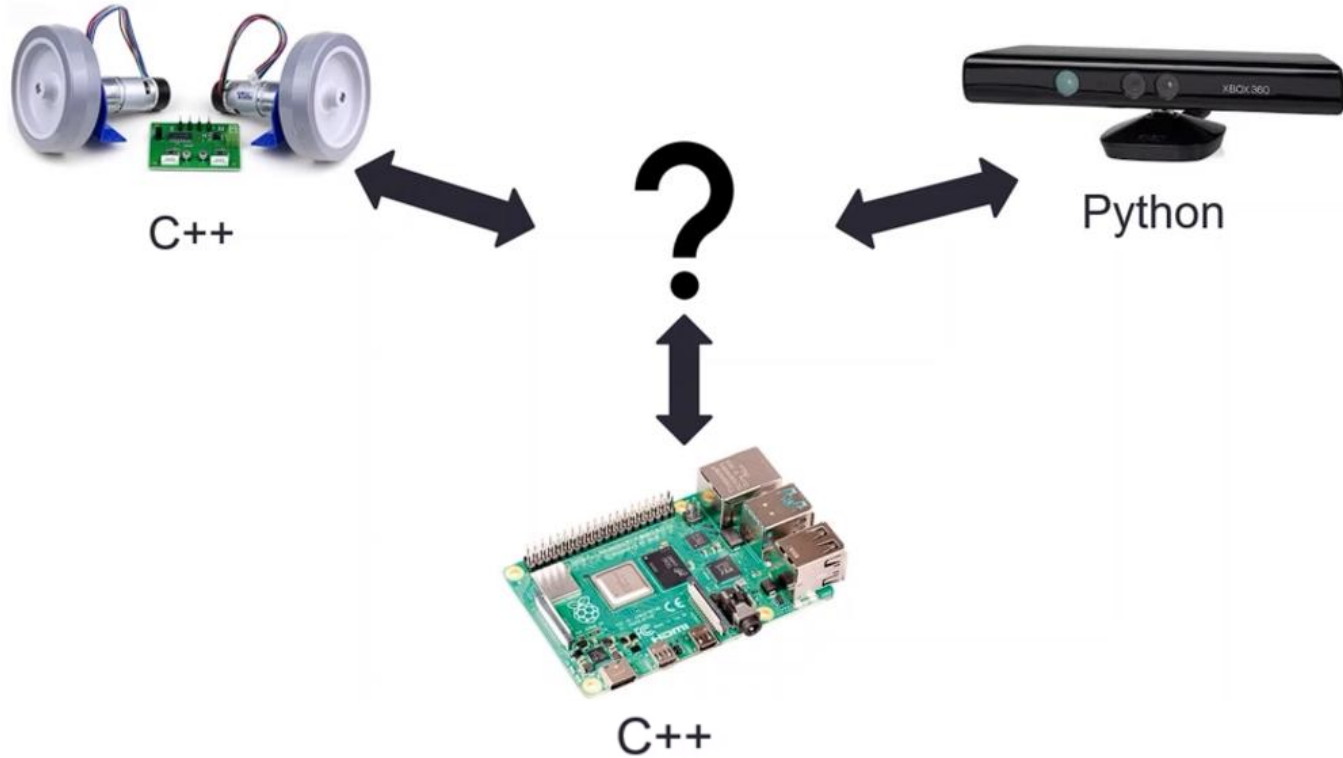


Python

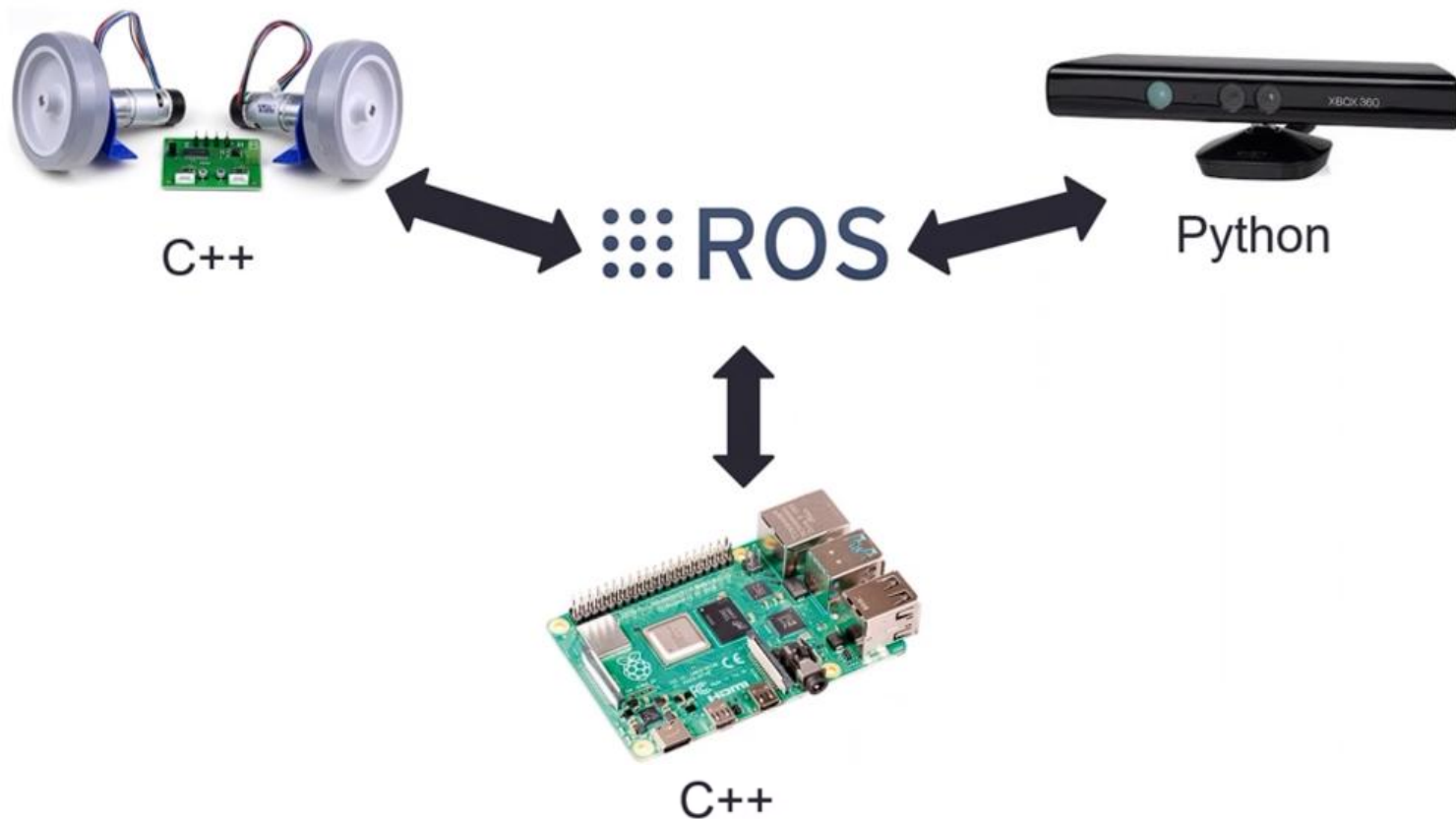


C++

Exemplo de Aplicação



Exemplo de Aplicação



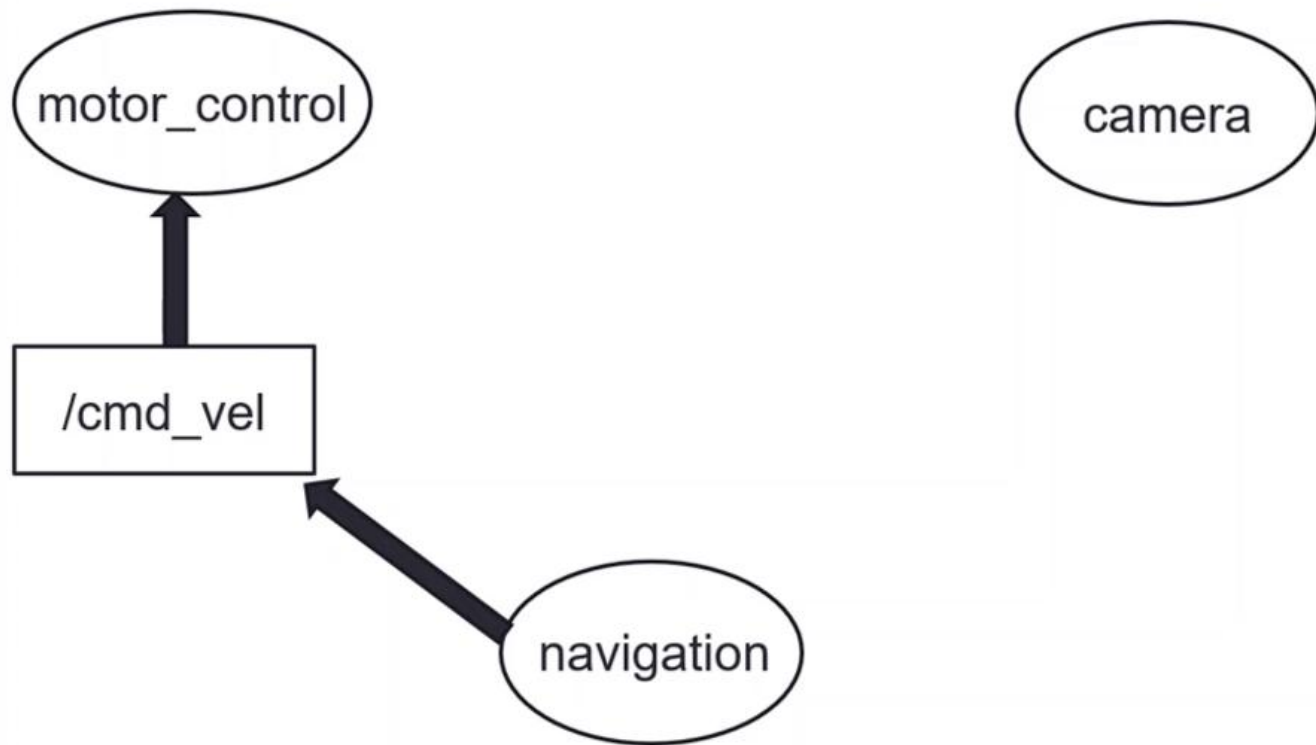
Exemplo de Aplicação

motor_control

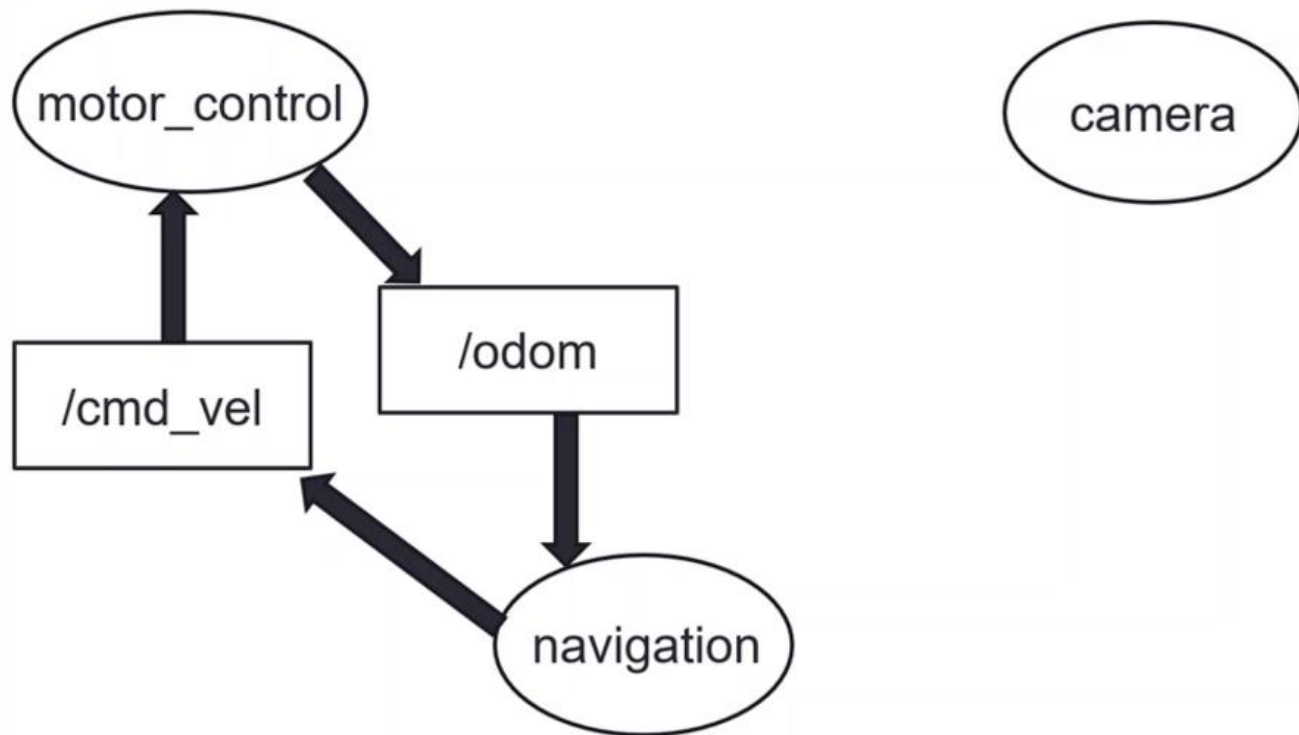
camera

navigation

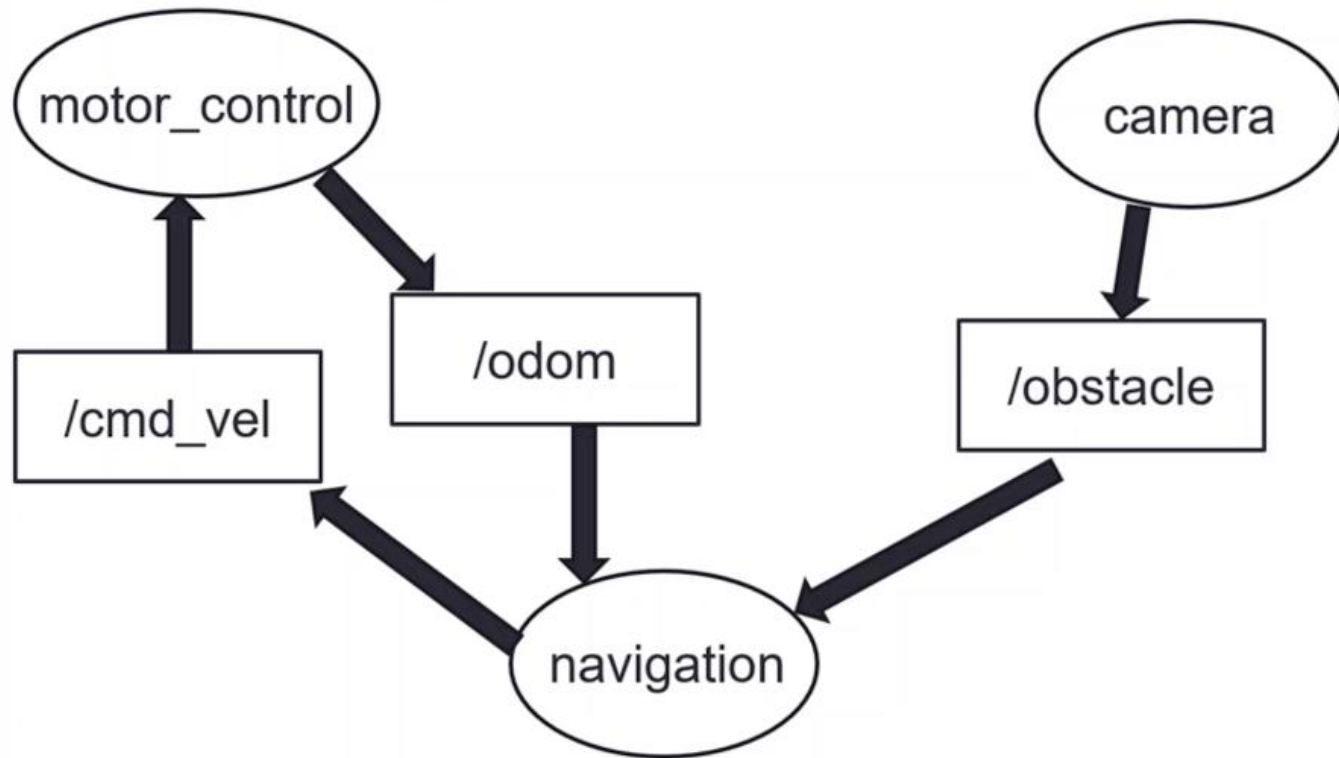
Exemplo de Aplicação



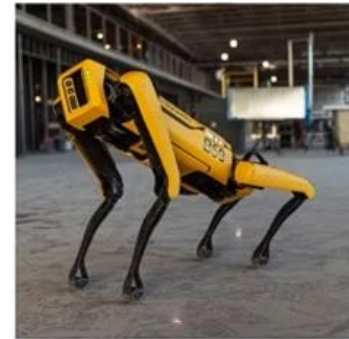
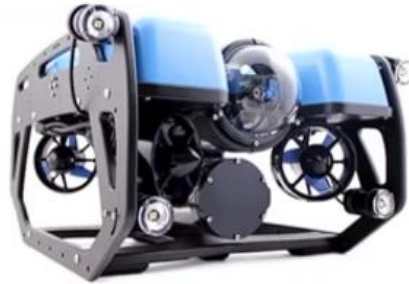
Exemplo de Aplicação



Exemplo de Aplicação



Robôs com ROS



Links Úteis

- ROS Wiki:
 - <https://wiki.ros.org/>
- ROS Tutorials:
 - <http://wiki.ros.org/ROS/Tutorials/>
- ROS Answer:
 - <https://answers.ros.org/>
- ROS Discourse:
 - <https://discourse.ros.org/>

Distribuições do ROS

- Padrão de lançamento do Ubuntu

Distro	Release date	Poster	Tuturtle, turtle in tutorial	EOL date
ROS Noetic Ninjemys (Recommended)	May 23rd, 2020			May, 2025 (Focal EOL)
ROS Melodic Morenia	May 23rd, 2018			May, 2023 (Bionic EOL)
ROS Lunar Loggerhead	May 23rd, 2017			May, 2019
ROS Kinetic Kame	May 23rd, 2016			April, 2021 (Xenial EOL)

Ubuntu 20.04

Ubuntu 18.04

Ubuntu 17.04

Ubuntu 16.04

Por que o ROS 2 existe?

O problema de desenvolver robótica sem framework

- **Heterogeneidade de hardware**
 - Sensores de fabricantes diferentes têm APIs incompatíveis
 - Trocar um LiDAR Hokuyo por um Velodyne = reescrever código do zero
- **Concorrência complexa**
 - LiDAR a 20 Hz, câmera a 30 Hz, controlador a 100 Hz
 - Gerenciar threads manualmente leva a condições de corrida
- **Distribuição frágil**
 - Raspberry Pi + GPU board + estação base precisam se comunicar
 - Sockets manuais: sem descoberta automática nem tolerância a falha
- **Reuso zero**
 - SLAM, Nav2, MoveIt2: anos de desenvolvimento para cada equipe

O que é o ROS 2

- **Framework de middleware para sistemas robóticos distribuídos**
- Não é um SO — é uma camada de software sobre Linux ou RTOS
- **Organização fundamental: o NÓ**
 - Processo independente com uma única responsabilidade
 - Exemplos: lidar_node / slam_node / motor_driver_node / camera_node
 - Se um nó trava, os outros continuam funcionando normalmente
- **Três mecanismos de comunicação padronizados**
 - Tópicos — publicação/assinatura contínua (LiDAR → SLAM)
 - Serviços — chamada síncrona request/response (ligar alarme)
 - Ações — tarefas longas com feedback e cancelamento (navegar)
- **Ecossistema de ferramentas**
 - RViz2 (visualização 3D) | RQt (diagnóstico) | Gazebo (simulação)

<https://docs.ros.org/>



LATEST ROS 2 LTS RELEASE

Lyrical Luth

Our latest long term support (LTS) ROS 2 distro, and the one we recommended for all ROS users.

[Documentation](#)

[Platform Support](#)



Jazzy Jalisco

A ROS 2 distro with support until May, 2029.

[Documentation](#)



Kilted Kaiju

A non LTS ROS 2 distro with support until November, 2026.

[Documentation](#)



Humble Hawksbill

A ROS 2 distro with support until May, 2027.

[Documentation](#)



Rolling Ridley

Our rolling release and the bleeding edge!

[Documentation](#)

Grafo de Computação e Middleware DDS

O Grafo de Computação

- **Conjunto de todos os nós e conexões ativas em tempo real**
- **Estrutura de um robô móvel típico:**
 - [lidar_node] —/scan—▶ [slam_node] —/map—▶ [nav_node]
 - [nav_node] —/cmd_vel—▶ [motor_driver_node]
- **Características:**
 - Dinâmico — nós sobem e descem sem afetar os demais
 - Cada nó declara o que publica, assina, oferece e consome
 - Visualização em tempo real: `rqt_graph`
- **Comandos CLI essenciais:**
 - `ros2 node list` → todos os nós ativos
 - `ros2 topic list` → todos os tópicos
 - `ros2 topic echo /scan` → mensagens em tempo real

ROS 1: roscore vs ROS 2: DDS

ROS 1 — Ponto único de falha

- **roscore: registro central obrigatório**
 - Todo nó consulta o roscore para se comunicar
 - roscore morreu → sistema inteiro para
 - Inaceitável em sistemas críticos
- **Impacto real:**
 - Robô cirúrgico: roscore cai → tudo para
 - Frota: um problema derruba todos os robôs

ROS 2 — Descoberta distribuída DDS

- **Não existe roscore no ROS 2**
 - Cada nó anuncia presença via multicast UDP
 - Nós se descobrem automaticamente, peer-to-peer
 - Um nó cai → os demais continuam operando
- **Responsabilidades do DDS:**
 - Descoberta automática sem configurar IPs
 - Serialização e transporte de mensagens
 - Políticas de QoS configuráveis por tópico

Isolamento: ROS_DOMAIN_ID e Namespaces

- **Problema: dois robôs na mesma rede WiFi interferem entre si**
 - /cmd_vel do Robô A chega no Robô B → comportamento imprevisível
- **Solução 1: ROS_DOMAIN_ID**
 - Inteiro 0–232 definido por variável de ambiente
 - DDS filtra: só chegam mensagens do mesmo DOMAIN_ID
 - `export ROS_DOMAIN_ID=1` # Robô A e sua estação base
 - `export ROS_DOMAIN_ID=2` # Robô B e sua estação base
- **Solução 2: Namespaces**
 - Organização lógica dentro do mesmo domínio
 - /robo1/cmd_vel vs /robo2/cmd_vel
 - Múltiplos robôs no mesmo grafo sem colisão de nomes de tópicos

Casos de Uso Reais do ROS 2

Sistemas Físicos com ROS 2

Robótica móvel e industrial

- **Robô de armazém (MiR, Amazon)**
 - SLAM + Nav2 + controle de motores
 - Frotas sem interferência via DOMAIN_ID
 - Algoritmos compartilhados — sem reescrever
- **Braço industrial (UR5, Franka)**
 - MoveIt2 + ros2_control
 - Ação FollowJointTrajectory com feedback
 - Cancelamento emergencial em tempo real

Veículos e sistemas críticos

- **Drone de inspeção (PX4 + ROS 2)**
 - Autopiloto via px4_ros_com
 - Nó de visão em GPU board separada
 - Detecção de fissuras em infraestrutura
- **Robô cirúrgico assistido**
 - Loop de controle a 1 kHz com PREEMPT_RT
 - DDS-Security: autenticação e criptografia
 - Sem roscore: zero ponto único de falha

Sistemas Simulados com ROS 2

Desenvolvimento e validação

- **Treinamento de algoritmos**
 - Gazebo publica /scan, /odom, /tf idênticos ao hardware
 - O algoritmo não distingue simulação de hardware real
 - Iterações rápidas sem risco de dano físico
- **Validação HIL (Hardware-in-Loop)**
 - Mesmo código no simulador e no hardware real
 - Só o driver de baixo nível é trocado

Integração contínua (CI/CD)

- **Testes automatizados a cada commit**
 - Gazebo headless em GitHub Actions / Jenkins
 - Robô executa rota → compara com resultado esperado
 - Build falha se comportamento divergir
- **Por que impossível no ROS 1:**
 - roscore precisa estar vivo em background
 - ROS 2: sem processo central = CI nativo

Comparar ROS 1 com ROS 2

Diferença 1 — Ponto único de falha vs Descoberta distribuída

Limitação do ROS 1

- **roscore: processo central obrigatório**
 - Todos os nós dependem dele para comunicar
 - Falha do roscore = falha de todo o sistema
- **Analogia: livro de endereços único**
 - Destruir o livro → ninguém se encontra mais
- **Impacto em produção:**
 - Robô cirúrgico: roscore cai → tudo para
 - Frota: um problema derruba todos os robôs

Solução do ROS 2

- **DDS com multicast UDP — sem roscore**
 - Cada nó anuncia presença ao subir na rede
 - Nós se descobrem peer-to-peer automaticamente
 - Falha de um nó → demais continuam operando
- **Impacto em produção:**
 - Robô cirúrgico: subsistemas continuam isolados
 - Frota: um robô cai → outros operam normalmente
 - CI/CD: sem roscore em background = testes nativos

Diferença 2 — Suporte a Tempo Real

Limitação do ROS 1

- **Linux padrão: agendador CFS não determinístico**
 - Callbacks podem ser atrasados indefinidamente
 - Loop de controle a 1 kHz = impossível garantir
- **Impacto em produção:**
 - Controlador de torque de exoesqueleto
 - Atraso de 5 ms → movimento brusco = risco de lesão
 - Frameworks externos necessários — fora do ROS 1

Solução do ROS 2

- **Suporte a RTOS e Linux com PREEMPT_RT**
 - Executor com prioridades de thread configuráveis
 - Janelas de execução determinísticas garantidas
 - ros2_control: interface padronizada para RT
- **Impacto em produção:**
 - Controlador de torque direto como nó ROS 2
 - Loop garantido a 1 kHz sem framework externo
 - Próteses e exoesqueletos com controle seguro

Diferença 3 — Segurança de Comunicação

Limitação do ROS 1

- **Comunicação não autenticada e não criptografada**
 - Qualquer nó na rede publica em qualquer tópico
 - Qualquer máquina chama qualquer serviço
- **Ataque possível em rede compartilhada:**
 - `rostopic pub /cmd_vel Twist "{linear:{x:1.0}}"`
 - Robô sai correndo — sem nenhuma autenticação
 - Inaceitável em ambientes industriais ou hospitalares

Solução do ROS 2 — SROS2

- **DDS-Security com SROS2**
 - Autenticação: certificados X.509 por nó
 - Autorização: quais nós publicam em quais tópicos
 - Criptografia: AES-128/256 em trânsito
- **Configuração:**
 - Arquivo de política XML por nó
 - Nó sem certificado = rejeitado pelo DDS
 - Integra com infraestrutura PKI corporativa

Quando ainda faz sentido manter o ROS 1?

- **Critério 1: Sistema legado estável + rede isolada**
 - Robô de pesquisa rodando ROS 1 Noetic há 3 anos sem problemas
 - Rede completamente isolada: sem risco de ataque externo
 - Equipe domina ROS 1: custo de migração supera o benefício
 - Conclusão: manter com plano de migração documentado
- **Critério 2: Dependência crítica sem port para ROS 2**
 - Ferramenta de pesquisa específica ainda sem equivalente em ROS 2
 - Manter temporariamente com plano de port definido
- **ROS 1 Noetic: suporte encerrado em maio de 2025**
- **Fora desses dois casos: qualquer sistema NOVO deve usar ROS 2**